- See previous practice problem sets for instructions.

1. For $n \in \mathbb{N}$ the n^{th} Fibonacci number F_n is defined by: $F_0 = F_1 = 1, F_i = F_{i-1} + F_{i-2}\ ;\ i \geq 2$.

   (a) Translate the following pseudocode for computing F_n into Python 3 code, and *time* it[1] on inputs $n = 35, n = 40, n = 45, n = 50$.

   SimpleFib(n)
   ```
   1  if n ≤ 1
   2        return 1
   3  F1 ← SimpleFib(n − 1)
   4  F2 ← SimpleFib(n − 2)
   5  return F1 + F2
   ```

   (b) Write the pseudocode for a *memoized* version of SimpleFib. Translate this pseudocode into Python 3 code, and time it on inputs $n = 35, n = 40, n = 45, n = 50$.

   Do you notice a significant difference? If you would like to *really* understand what is going on here: Try adding print statements to inspect the arguments passed in to the recursive calls in your memoized version.

   (c) Derive a good asymptotic upper bound in terms of n, on the running time of your pseudocode from part (b).

2. Consider the problem of computing the number of ways to climb a staircase with n steps, where you can take either 1 step or 2 steps at a time.

   (a) Write the pseudocode for a *recursive* function Ways(n) which implements the above logic to compute and return the number of ways of climbing a staircase with n steps. Use the recurrence:

   $$\text{Ways}(n) = \text{Ways}(n-1) + \text{Ways}(n-2),$$

   with base cases $\text{Ways}(0) = 1$ (one way to stay at the bottom) and $\text{Ways}(1) = 1$.

   (b) Translate your pseudocode of part (a) into Python 3 code, and time it on inputs $n = 20, n = 30, n = 40, n = 50$.

   (c) Write the pseudocode for a *memoized* version of the recursive function from part (a). Implement this memoized version in Python 3, and time it on the same inputs as above. What improvements do you observe?

   (d) Derive a good asymptotic upper bound in terms of n, on the running time of your pseudocode from part (c).

   (e) Add print statements to the memoized version to track which values of n are being computed during the recursion. Does this help you understand how the memoization is speeding things up?

[1] That is: find the clock-time taken by the code.

3. For non-negative integers n, k ; $k \leq n$ the binomial coefficient $\binom{n}{k}$ represents the number of ways to choose k items from n items without regard to order. It satisfies the recurrence:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k},$$

with base cases $\binom{n}{0} = \binom{n}{n} = 1$.

(a) Write the pseudocode for a *recursive* function Choose(n, k) which implements the above logic to compute and return $\binom{n}{k}$.

(b) Translate your pseudocode of part (a) into Python 3 code, and time it on inputs $(n, k) = (20, 10), (30, 15), (40, 20), (50, 25)$.

(c) Write the pseudocode for a *memoized* version of the recursive function from part (a). Implement this memoized version in Python 3, and time it on the same inputs as above. What improvements do you observe?

(d) Derive a good asymptotic upper bound in terms of n, on the running time of your pseudocode from part (c).

4. Consider the problem of finding the minimum *number* of coins required to make change for a given integer amount $n \geq 0$, using coins of fixed positive integral denominations $d_1, d_2, \ldots, d_k$.

(a) Derive a recurrence relation for this problem. That is: let MinCoins(n) denote the minimum *number* of coins that add up to exactly the value n, where each coin takes one of the values ("denominations") $d_1, d_2, \ldots, d_k$, and there can be zero or more coins with the same denomination. Make sure that you capture all the relevant base cases of your recurrence.

Explain why your recurrence correctly computes MinCoins(n).

(b) Write the pseudocode for a *recursive* function MinCoins(n) which implements the above logic to compute and return MinCoins(n).

(c) Translate your pseudocode of part (b) into Python 3 code, and time it on the following inputs:

- The denominations are $d_1 = 1, d_2 = 5, d_3 = 10, d_4 = 25$ for all the values.
- The values are: $n = 20, n = 50, n = 100, n = 200$
- (Thus there are four instances, one for each value of n.)

(d) Write the pseudocode for a *memoized* version of the recursive function from part (b). Implement this memoized version in Python 3, and time it on the same inputs as above. What improvements do you observe?

(e) Derive a good asymptotic upper bound in terms of n, on the running time of your pseudocode from part (d).

5. Consider the following problem. You are given an array $A[0 \ldots n-1]$ of n integers (which may be positive, zero, or negative). Your task is to select a subset of elements from A such

that no two selected elements are *adjacent* in the array, and such that the *sum* of the selected elements is as large as possible.

In other words, if you choose the element at index i, then you are not allowed to choose the elements at indices $i-1$ or $i+1$.

For example:

- If $A = [3, 2, 5, 10, 7]$, then the maximum possible sum of non-adjacent elements is $3 + 5 + 7 = 15$.
- If $A = [5, 5, 10, 100, 10, 5]$, then the maximum possible sum is $5 + 100 + 5 = 110$.
- If $A = [4, -1, 2, 0, 3]$, then the maximum possible sum is $4 + 2 + 3 = 9$.
- If $A = [-2, -3, -1]$, then the maximum possible sum is 0, obtained by selecting no elements at all.

Let MaxNonAdj(i) denote the maximum sum that can be obtained using only the subarray $A[0 \ldots i]$, under the restriction that no two chosen elements are adjacent.

(a) Derive a recurrence relation for MaxNonAdj(i). Make sure that you clearly specify all relevant base cases.

Explain why your recurrence correctly computes the maximum sum of non-adjacent elements of $A[0 \ldots i]$.

(b) Write the pseudocode for a *recursive* function MaxNonAdj(i) which implements the above recurrence to compute and return MaxNonAdj(i).

(c) Translate your pseudocode of part (b) into Python 3 code, and time it on the following inputs:

- Arrays of sizes $n = 20, 30, 35, 40$.
- For each n, generate an array of length n whose entries are chosen uniformly at random from the range $[-50, 50]$.

What do you observe about the running time as n increases?

(d) Write the pseudocode for a *memoized* version of the recursive function from part (b). Implement this memoized version in Python 3, and time it on the same inputs as above. What improvements do you observe?

(e) Derive a good asymptotic upper bound, in terms of n, on the running time of your pseudocode from part (d).